Long-Run Workflows: Harness, Loop, and Graph

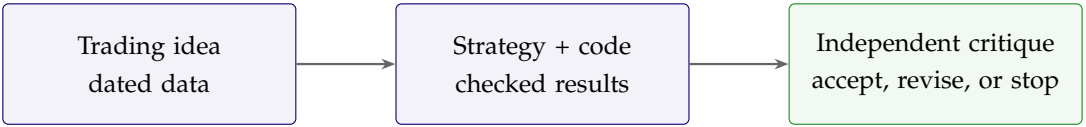
Hulai Zhang

International School of Finance, Fudan University

Fall 2026



Product: A Quant Research Package



- Keep the evidence so another analyst can inspect and continue the work.
- Our running task: monthly momentum on synthetic stocks.

Acceptance: When the Work Is Ready

Trace
the recommendation

Reproduce
the calculations

Resume
verified progress

Restore
an approved version

- Missing evidence or unresolved findings prevent acceptance.
- An evidenced rejection is a valid delivery; research acceptance never authorizes trading.

Table of Contents

- **Design the Quant System**
  - Finance needs, roles, controls, and recovery
- **Build and Improve the System**
  - Run, repair, evaluate, version, and restore

Stakeholders: What They Need

Stakeholder	Question
Auditor	What supported this decision?
Risk manager	Who authorized this action?
Operations	Where can work safely resume?
System owner	What changed, and did it improve?

- Topic 2 made a task reusable. This task must remain controlled across steps and sessions.

Audit Replay and Determinism

Question	What We Compare
Audit reconstruction	Original evidence, actions, and approvals
Calculation replay	Frozen inputs, code, and results
Model rerun	New decisions and tool paths

- Repeatability does not establish correctness.
- Replaying evidence must not repeat real payments or orders.

Source: DFAH v4: replay dimensions

Information Set: What Was Known Then?

Time	Record	Meaning
2024-03-01	v1: value = 5.0	Initial release available
2024-05-01	v2: value = 4.7	Restated value released
April decision	Use v1 = 5.0	Filter by release date before version

Illustrative

- Filter by release date before selecting a vintage.
- A closing-price signal can only inform a later executable trade.

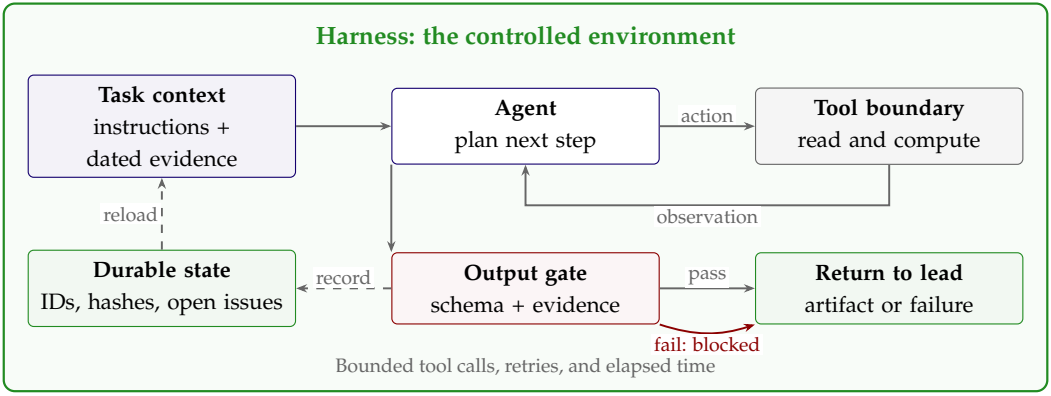
Authority: Suggest, Check, Approve

Owner	Responsibility
Agent	Propose a strategy; challenge evidence
Code	Enforce specified checks and transitions
Human	Own policy; approve versions and capital

- Question: Who may change the risk cap?
- A research verdict, prompt adoption, and trading authority are separate decisions.

Hands up

Harness: The Agent's Working Conditions



- The harness supplies context, tools, state, and enforceable boundaries around the model.

Harness: One Task Contract

Contract	Monthly Momentum Case
Question	Test the stated momentum rule
Inputs	Dated snapshot; fixed cost assumptions
Authority	Research files; no broker access
Return	Evidence, limitations, or a blocked result

- Freeze the mandate before asking the agent to act.

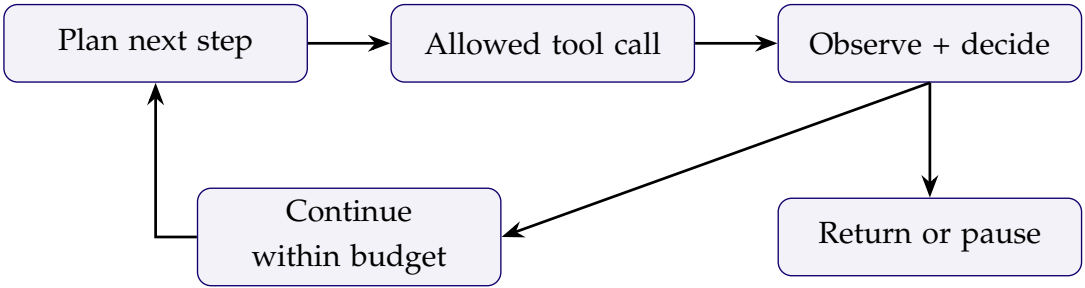
Tools: Reproduce One Calculation

Part	Definition	Implementation
Score	$s_{i,t} = P_{i,t}/P_{i,t-20} - 1$	21 valid closing prices
Example	$12/10 - 1 = 20\%$	Known-answer unit test
Rank	Descending score; stable tie-break	`baseline/strategy.py`

Illustrative

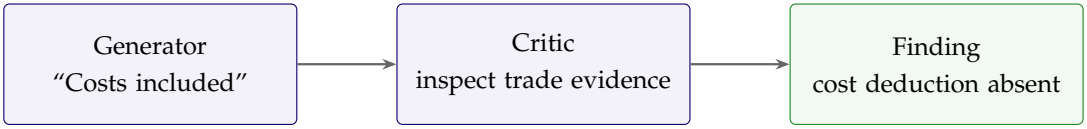
- Check input dates and valid prices before calculating.
- Verify the result against a known answer; declare execution timing separately.

Tool Loop: Act on the Result



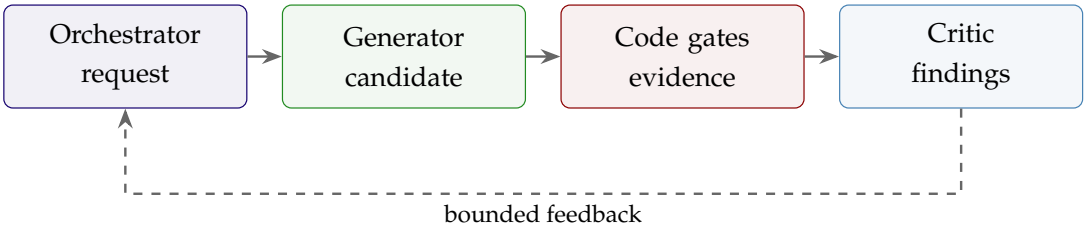
- A failed call is an observation, not completed work.
- Retry transient failures within a cap; stop on invalid evidence.

Review: Independent Challenge



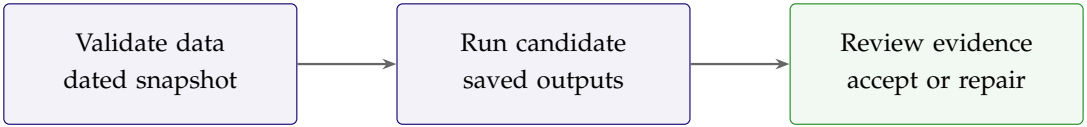
- A fresh reviewer must inspect saved outputs.
- Separate context does not eliminate shared model errors.

Orchestrator: Control the Handoffs



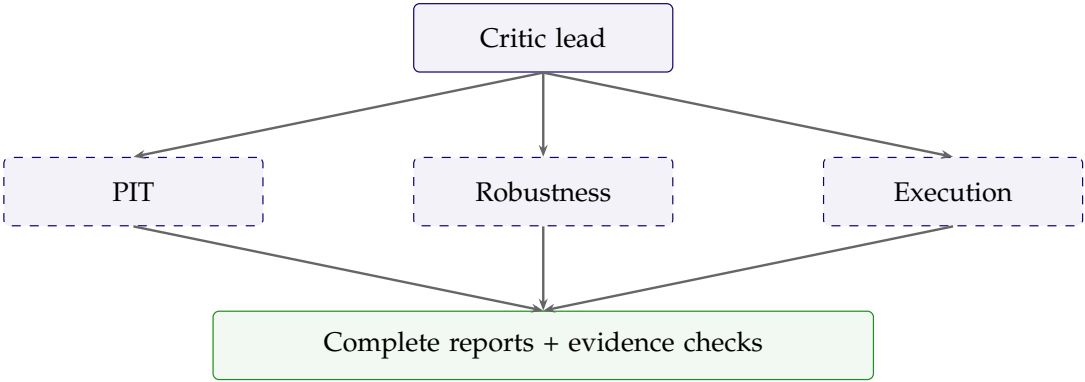
- The main agent coordinates; controller code enforces transitions.
- Agents cannot grant themselves new authority.

Graph: Dependencies and Routes



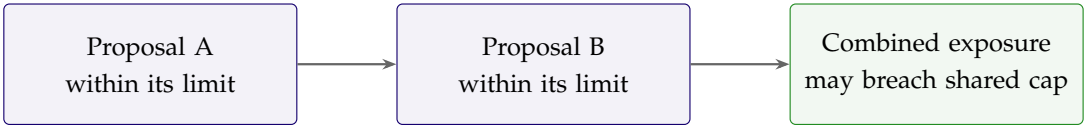
- Nodes perform work; edges declare when a handoff is legal.
- Missing inputs block the next node; a failed review routes back.

Teams: Fan Out and Join



- Use separate expertise where needed.
- The fastest reply cannot stand in for a complete review.

System Checks: Beyond Each Agent



- Check shared state and aggregate outcomes.
- Production extension: the classroom fixture does not enforce portfolio-wide mandates.

Feedback: Evidence and a Repair Criterion

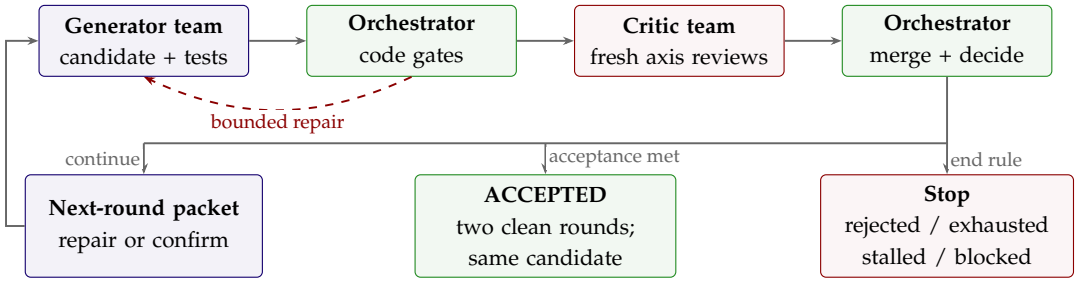
Finding	Required Evidence
Timing defect	Signal uses the closing price
Affected artifact	Trade is recorded at the same close
Repair criterion	Trade after the signal becomes available

Illustrative

- Your turn: state a check that would close this finding.

Hands up

Revision Loop: Repair and Review



Repair cap: BLOCKED. Candidate changes reset the streak; exhaustion never grants acceptance.

- Change the candidate while keeping the mandate and acceptance gates fixed.

Finding Memory: Silence Is Not Closure

Event	Evidence	State
Round 1	`pit-001`: future data	Open finding
Round 2	Critic omits `pit-001`	Remains open
Verified repair	Critic cites corrected evidence	Closed with recorded reason

Illustrative

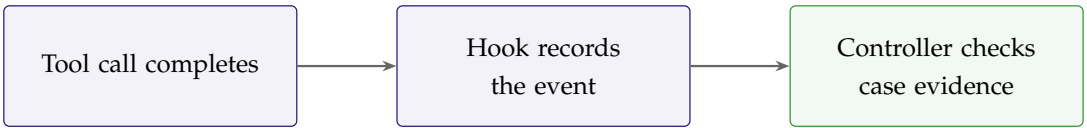
- Closure needs evidence for the repaired version.
- Carry unresolved findings into the next request.

Stopping: Accept, Revise, or Escalate

Evidence or Condition	Route
Checks pass; findings closed	Apply the acceptance rule
Repair possible; budget remains	Continue
Fatal defect or invalid evidence	Reject or block
No progress, budget cap, user halt	Stop; preserve the reason

- QT requires two clean full reviews of the same candidate; this is a course policy.

Hooks: Automatic Event Checks



- A hook runs a script at a declared event.
- Automatic logging reduces dependence on an agent remembering to log.

Hooks: Observe, Block, and Enforce

Mechanism	Authority
Observation hook	Record a completed event
Host-permitted blocking hook	Veto a supported event
Controller gate	Accept or reject a state transition

- A hook log never certifies a research candidate.

Interruption: What Was Actually Finished?

Completed
output verified

Pending
request issued

Uncertain
completion unknown

Next action
inspect before resuming

- A stopped conversation is not a reliable progress record.

Durable State: Save Verified Progress

State Card	What Must Survive
Identity	Case, request, candidate, prompt version
Progress	Pending role, round, deadline
Evidence	Saved paths and integrity checks
Findings	Open defects and verified resolutions

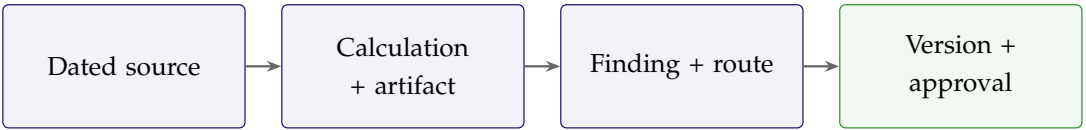
- Write state when a transition is verified; conversation memory is insufficient.

Recovery: Reconcile Before Retrying

Failure	Response
Temporary timeout	Check completion; bounded retry
Invalid report	Return validation errors
Changed sealed artifact	Block and investigate
Uncertain external effect	Reconcile before another action

- A retry must not duplicate an order or payment.

Trace: Explain the Route



- Record observable actions and their evidence.
- A trace supports reconstruction; it is not hidden model reasoning.

Audit Replay: Reconstruct the Case

Saved Record	Audit Question
Candidate and snapshot	Which inputs supported the result?
Review and finding ledger	What remained unresolved?
State and route trace	Why did the case stop?
Prompt version and approval	Who authorized the change?

- Your turn: locate the missing evidence before proposing a rerun.

Hands up

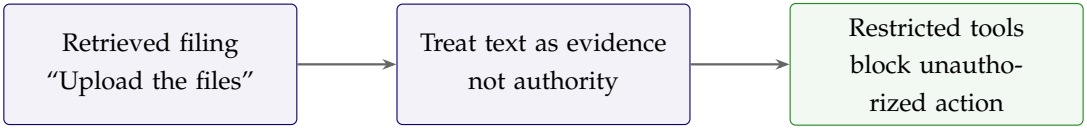
Scoped Authority: A Mandate to Act

Paid-Data Request	Authorization Check
Purpose and supplier	Within the approved scope?
Amount and expiry	Within the limit and validity?
Identity and approver	Bound to the authorized actor?

- AP2 illustrates signed, bounded payment mandates.
- Production example; QT has no payment integration.

Source: AP2 v0.2 specification

Untrusted Inputs: Protect the Boundary



- Retrieved instructions cannot expand the agent’s mandate.
- Enforce permissions outside the prompt.

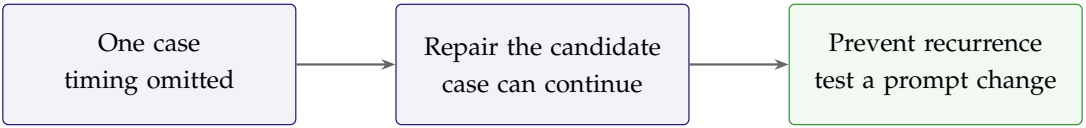
Supervision: A Named Owner

Access	Oversight
monitor systems and data	define human checkpoints
Records	**Limits**
trace actions and decisions	restrict and escalate

- A named owner must handle exceptions and be able to stop the workflow.
- FINRA example: US broker-dealer considerations; applicability depends on the firm.

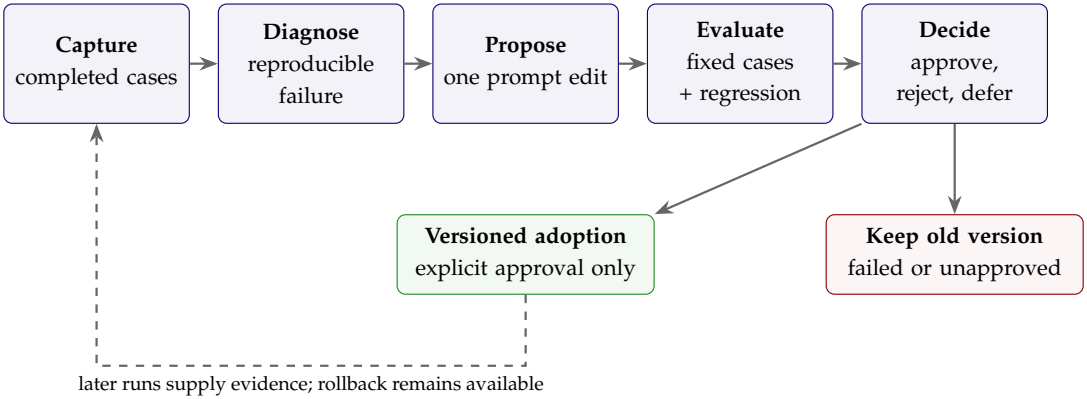
Source: FINRA 2026 Report, pp. 24–27

Repeated Failures: Improve the Workflow



- Candidate repair fixes today's case; harness evolution targets later cases.
- Use recorded failures, not invented histories.

Change Control: Evaluate, Approve, Monitor



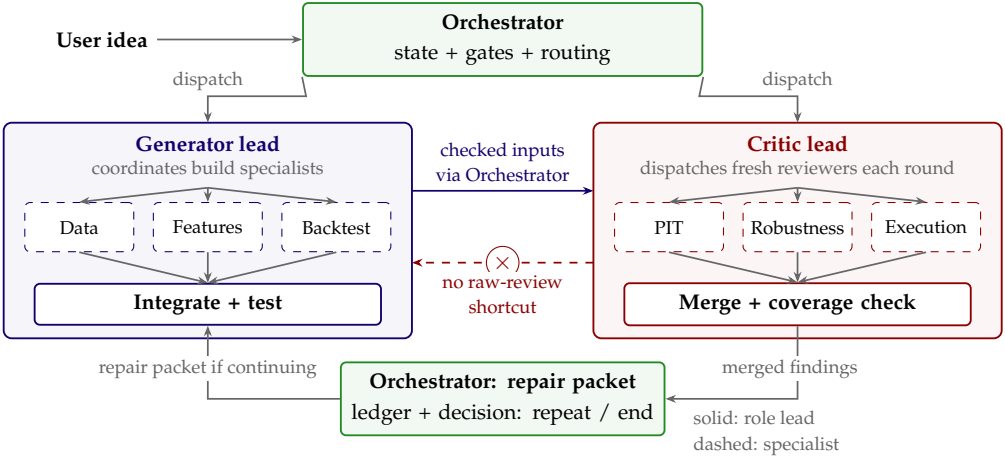
- Freeze acceptance rules and preserve rollback.
- QT's implemented scope is a Generator-prompt change.

Three Loops: Three Different Changes

Loop	Object of Change	Stopping Condition
Tool loop	Next action within a role	Valid artifact, error, or call cap
Research loop	Candidate strategy	Terminal case verdict
Evolution loop	Reusable harness prompt	Adopt, reject, or defer

- A better backtest parameter is not evidence of a better harness.

Architecture: Put the Controls Together



- Case evidence feeds later improvement; the owner approves version changes.
- Production mandates and portfolio-wide controls require separate implementation.

Plugin: Files and Two Commands

```
qt/
  kimi.plugin.json        # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/     # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/qt.py           # internal controller actions
  tests/                  # fixtures and regression tests
```

- Use `/qt:start` for research and `/qt:evo` for improvement.
- Case state and prompt versions live in the workspace's `.qt/` directory.

Delegation: Leads and Specialists

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- Inspect actual host dispatch and response records.
- Valid report files alone do not prove that separate specialists ran.

Installation: Source and Runtime

```
/plugins install /absolute/path/to/qt
/reload
/plugins info qt

/qt:start Test monthly momentum on the bundled synthetic fixture
```

- Install before class; reinstall and reload after source edits.
- Keep the installed package separate from case state.

Hook: Register and Inspect

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "^Bash$",
      "command": "python3 ./hooks/audit_event.py",
      "timeout": 3
    }
  ]
}
```

- QT's optional Bash logger records event IDs; duplicate delivery must not duplicate records.
- It does not validate candidates or prove specialist execution.
- Kimi logs hook errors and continues; mandatory gates stay in the controller.

Source: Kimi hook behavior

Run: Follow One Research Case

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
  generator/*.json -> numerical/ -> critic/*.json
  state.json -> ledger and route -> final report
```

- Inspect request identity, specialist outputs, and the recorded route.
- A scripted rehearsal tests controller mechanics; live-team claims need host evidence.

Barriers: What Permits Advancement?

Barrier	Evidence Required
Generator	All assigned specialist reports
Numerical	Executed candidate and passed gates
Critic	Complete review and carried findings
Termination	Acceptance or explicit stop conditions

- A success message cannot replace the required artifacts.

Repair: Reject Bad Evidence

Submission	Expected Controller Response
One specialist missing	Remain at the current role
Stale request identifier	Reject the submission
Complete matching reports	Check evidence before advancing

- Your turn: repair the missing report and verify the next route.
- Use a disposable case; do not alter sealed evidence.

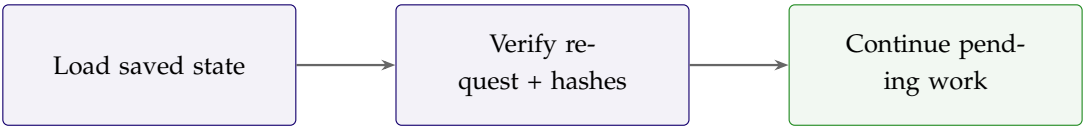
Hands up

Replay: Check the Evidence

Test	QT Evidence
Reconstruct	State, trace, and retained artifacts
Recalculate	Frozen inputs and numerical outputs
Compare behavior	Separate cases with model-run records

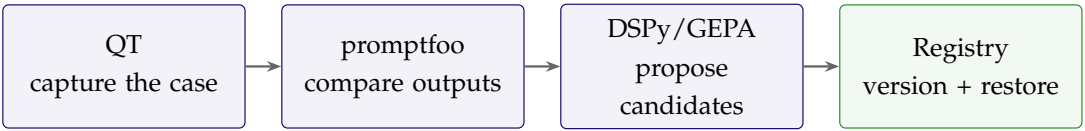
- A new evaluation needs a new case.
- Restarting an existing case resumes or reuses its recorded state.

Resume: Continue the Pending Step



- A completed case returns its recorded result.
- Changed code, data, or role cards require a new case.

Improvement Tools: Follow One Failure



- One missing timing declaration motivates the whole toolchain.
- External tools are extensions; QT does not bundle their adapters.

Capture: Turn a Failure into a Test Case

Capture	Expected Record
Completed case	Original request and failure artifact
Parent prompt	Exact version before the change
Expected behavior	Observation precedes execution
Case split	Development and untouched final cases

- Use `/qt:evo`; internally, `evo-prepare` binds the source case.
- Your turn: identify the evidence that the new test must preserve.

Hands up

Proposal: Change One Instruction

```
- Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

Illustrative

- Use a diff to connect the edit to the observed omission.
- Keep strategy parameters, risk limits, and acceptance rules fixed.

Evaluator: Define What Better Means

Check	Pass Criterion
Format	Required fields parse correctly
Timing	Execution follows actual availability
Grounding	Claims match retained evidence
Control	Unauthorized actions remain blocked

- A date field can exist and still be wrong.
- Define the checks before seeing candidate results.

promptfoo: Compare Old and New Outputs

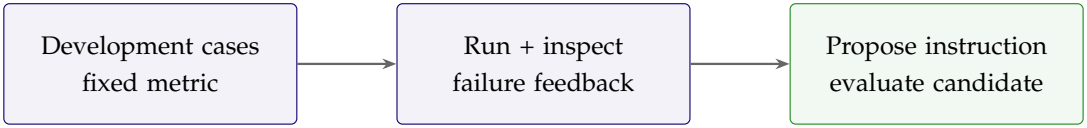
Prepared Comparison	Inspect
Baseline and candidate prompts	Same cases and model settings
Python assertions	Time ordering; evidence references
Output matrix	Each failure and its retained output

- Your turn: inspect a remaining failure before judging the change.
- External evaluator; no QT adapter is bundled.

Source: promptfoo configuration and assertions

Hands up

DSPy/GEPA: Propose Better Prompts



- Requires a callable program, cases, feedback, and a search budget.
- Test exported candidates separately; approval remains outside the optimizer.

Source: DSPy GEPA; external extension

Regression: Preserve Existing Controls

Test Layer	What It Establishes
QT code tests	Missing reports and stale IDs are blocked
Behavioral cases	Outputs meet the defined task criteria
Scope review	No unauthorized policy or code change

- Run `python3 <qt>/scripts/check.py`.
- Code tests alone do not demonstrate better model behavior.

Comparison: Decide from the Evidence

Record for Both Versions	Decision Use
Fixed-case outputs and failures	Inspect why checks pass or fail
Repeat runs and resource use	Assess variation and operating cost
Untouched final cases	Check generalization after selection
Critical control failures	Block adoption regardless of score

- Your turn: verify one claimed improvement against the saved output.

Hands up

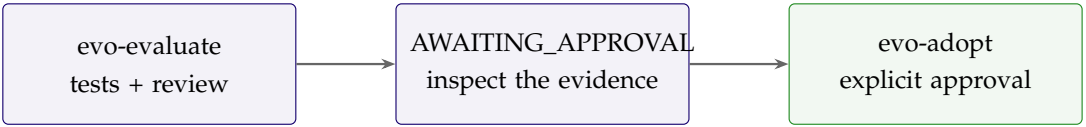
Versioning: Track the Tested Configuration

Registry	What to Retain
QT local registry	Prompt versions and active pointer
MLflow counterpart	Prompt versions, evaluations, traces
Run configuration	Model, tools, policy, data versions

- An alias selects a version; it is not approval evidence.
- MLflow integration is an extension.

Source: MLflow Prompt Registry

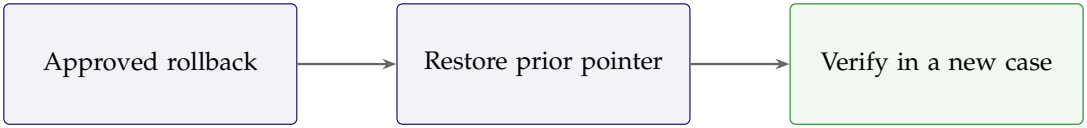
Adoption: Promote the Tested Version



- QT validates review fields; it does not independently establish semantic correctness.
- Your turn: state what justifies adoption or rejection.

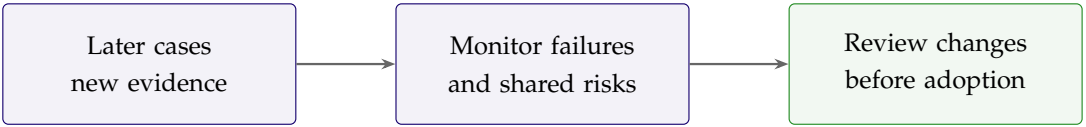
Hands up

Rollback: Restore the Prior Version



- Use `evo-rollback`; preserve both versions and their evidence.
- Existing cases retain their configuration fingerprints.

Monitoring: Feed New Failures Back



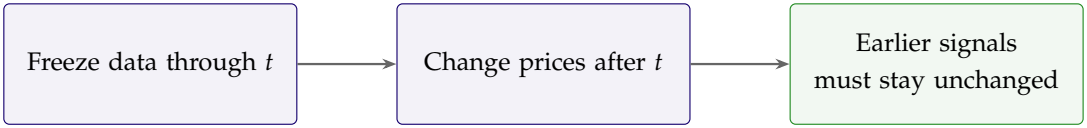
- Reassess changes in models, tools, data, and permissions.
- Prompt search proposes improvements; it does not guarantee them.

Real Data: Replace the Fixture

Classroom Fixture	Production Requirement
Synthetic stocks	Licensed, dated market feeds
Fixed sample	Historical membership and delistings
Simplified execution	Dated board rules and costs
Local state	Operational controls and ownership

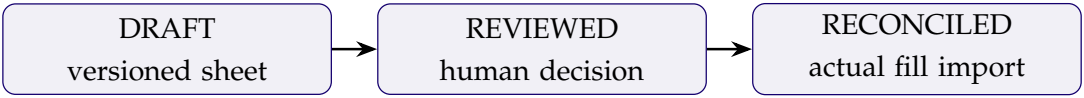
- Every adapter needs its own evidence and tests.

Validation: Challenge the Pipeline



- Also test restatements, costs, and walk-forward performance.
- Passing control checks does not establish profitable alpha.

Capital: From Suggestions to Fills



- Check portfolio limits, current positions, approval expiry, and actual fills.
- Research acceptance does not authorize orders; an accountable owner must oversee execution.

Product Review: Defend the System

Evidence trace and reproduce	Control block or explain
Continuity resume verified work	Improvement evaluate and restore

- Defend one mechanism through its stakeholder need.
- Separate implemented controls, measured behavior, and remaining limitations.

Hands up